COSC419/519:DEEP LEARNING

Linear Models

Part-2

Dr. Mohamed Shehata

Computer Science

University of British Columbia

UBC OKANAGAN

Multi-class Regression

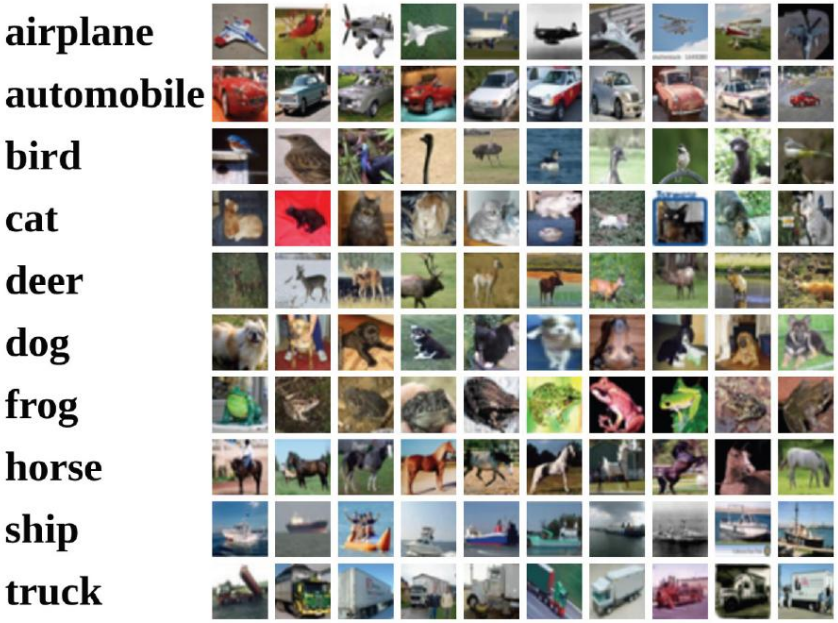
Two-Class vs Multi-Class

- In many applications, we need to classify multiple classes (C>2) and not only two
  - Number digits 0 to 9
  - Letters
  - Hand gestures…
- We can generalize the two-class classifiers we learned into multi-class classifiers

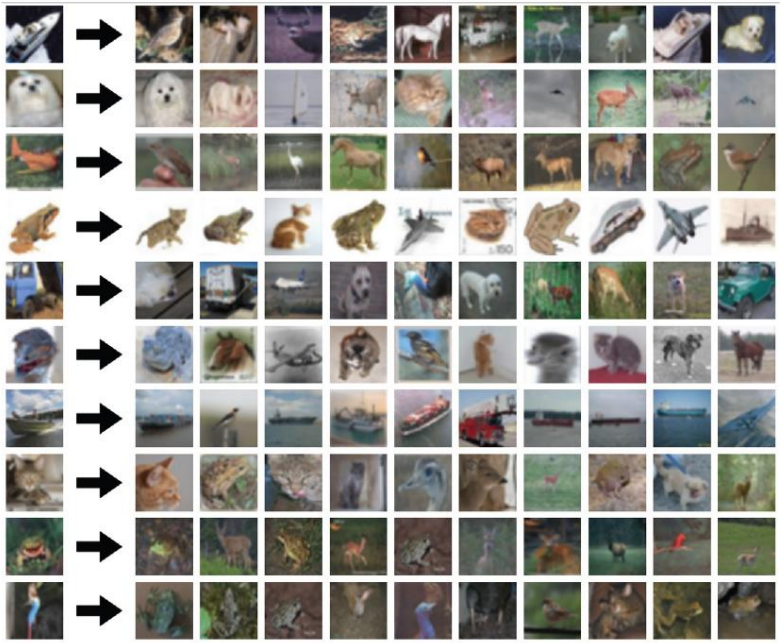
Multi-class Classification Example

Example Dataset: CIFAR10

10 classes
50,000 training images
10,000 testing images



Test images and nearest neighbors



Alex Krizhevsky, "Learning Multiple Layers of Features from Tiny Images", Technical Report, 2009.

Example

Image x

$f(\boldsymbol{w}, \boldsymbol{x_p})$

10 numbers giving class scores

Array of **32x32x3** numbers (3072 numbers total)

parameters or weights

Example Problem

- We want to classify Iris flowers (Setosa, versicolor, virginica) using only sepal length and width
- We have access to labelled measurements of sepal length and width
- What is N and C in this case?

General Setup

Input $\{(\boldsymbol{x}_p, y_p)\}_{p=1}^{P}$
N=2: length, width
e.g., $(x_{p,1}, x_{p,2})$of the p data

Output:
class $\hat{y}_p \in \{0,1,2\}$,
$p = 1,2, \ldots, P$
C=3 (setosa, versicolor
, and verginica)

Data set

Then during testing

New data:
length,width (x_1, x_2)

Learning
(or training)

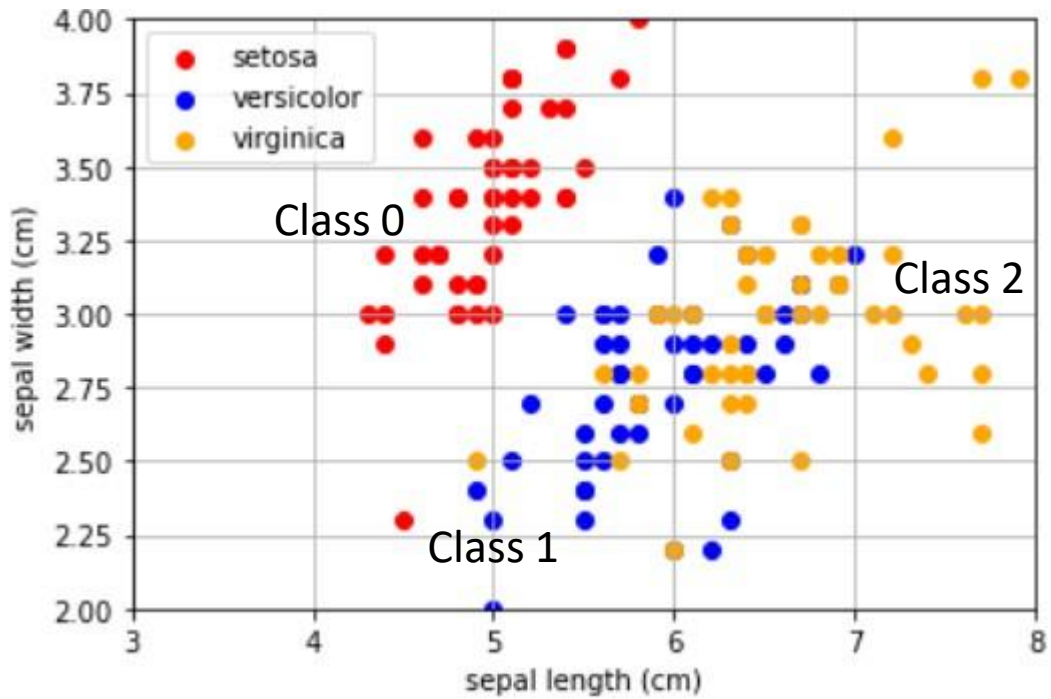
Classification model
$f(w, x_p)$

Tuning the parameters ($\boldsymbol{w}$)
of the model

Predicted class
$\widehat{y_p} \rightarrow 0 =$ setosa
$\widehat{y_p} \rightarrow 1 =$versicolor
$\widehat{y_p} \rightarrow 2 =$verginica

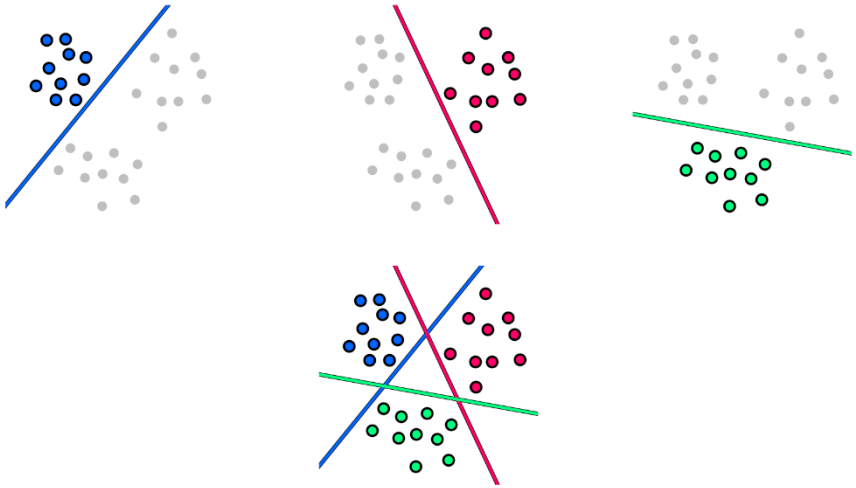
Multi-Class Dataset

- Consists of data points $(\boldsymbol{x}_1, y_1)$, $(\boldsymbol{x}_2, y_2)$,…, $(\boldsymbol{x}_P, y_P)$ where
  - $\boldsymbol{x}_p = (x_{p,1}, x_{p,2}, \dots, x_{p,N})$ is an N-dimensional input vector *(feature vector)*
  - y_p is the class label, $y_p \in \{0,1, \dots, C-1\}$ where C is the number of classes
- In our case
  - $\boldsymbol{x}_p = \left(x_{p,1}, x_{p,2}\right)$ for sepal length and width
  - y_p is the class label, $y_p \in \{0,1,2\}$ for C=3 {setosa, versicolor, and verginica}
- Our task is to build a classifier that can distinguish between the 3 classes

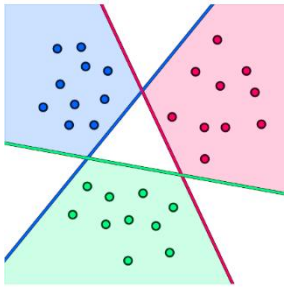


One-versus-All classifiers

- Learn C two-class classifiers on the entire dataset (with the cth classifier trained to distinguish the cth class from the remainder of the data).
- The three places a point in the space can end up (w.r.t our classifiers):
  - A) On the positive side of a single classifier
  - B) On the positive side of more than one classifier
  - C) On the negative side of all classifiers



Points on the positive side of a single classifier



- Denoting the weights from the cth classifier as:

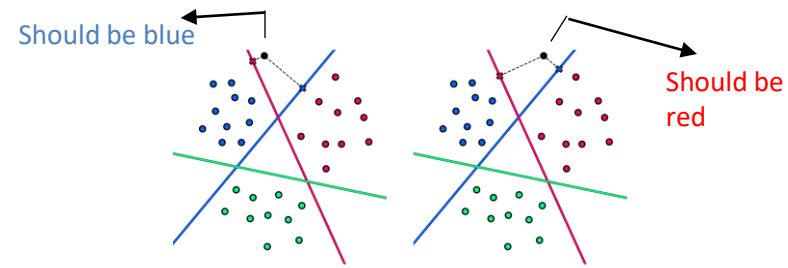
$$w_c = \begin{bmatrix} w_{0,c} \\ w_{1,c} \\ w_{2,c} \\ \vdots \\ w_{N,c} \end{bmatrix}$$

- We can write the corresponding decision boundary: $x^T w_c = 0$
- For N=2, C=3, we have x^T is size 1x3 and w_c is size 3x1 and hence $x^T w_c$ is size 1x1 which is a scalar which is $\hat{y}$ using this particular classifier
- Note that
  - A point on the positive side of classifier j has: $x^T w_j > 0$ (positive value)
  - A point on the negative side of classifier j has: $x^T w_j < 0$ (negative value)
- The points that lie solely on the positive side of the jth classifier (w_j) should clearly belong to the jth class, so they satisfy $x^T w_j = \max_{c=0,1,\ldots C-1} x^T w_c$
- Therefore, to get the label $\hat{y}$ associated with that point, we use:

$$\hat{y} = \underset{c=0,1,\ldots C-1}{\operatorname{argmax}} \; x^T w_c$$

Points on the positive side of more than one classifier

- **Classification rule:** in such regions we assign each point to the class whose boundary is at the largest nonnegative distance from that point.



Proof:

- When **multiple classifiers** say this (point is positive for multiple classifiers), we have a conflict

- Why pick the largest nonnegative distance?
  - The value $x^T w_c$ represents how "confident" classifier j is about the point belonging to class j
    - Larger positive values = point is further into the positive side = more confident
    - Smaller positive values = point is closer to the boundary = less confident
  - Then this distance is simply written as the raw evaluation of the point via the decision boundary:

$$\text{Signed distance of } x \text{ to } j^{th} \text{ boundary} = x^T w_j$$

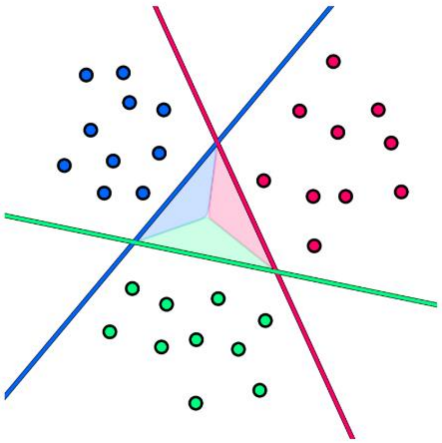
  - This satisfy $x^T w_j = \max_{c=0,1,\ldots C-1} x^T w_c$

- So again, after weight-normalization, we have the same prediction rule that we found originally for regions of the space where only a single classifier is positive

$$\hat{y} = \underset{c=0,1,\ldots C-1}{\operatorname{argmax}} \; x^T w_c$$

Points on the negative side of all classifiers

- In this case we cannot argue that one classifier is more 'confident' in the class identity of such points. However, we can ask which classifier is the least 'unsure' about the class identity of such points
- Since all classifiers are saying "this point is NOT in my class," we apply a different logic:
  - **Pick the classifier with the LEAST NEGATIVE value** (i.e., the largest among negative values)



- Why this makes sense:
  - $x^T w_j$ = - 0.1: "I'm barely on the negative side, just slightly outside my region"
  - $x^T w_j$ = - 5.0: "I'm very far into the negative side, definitely NOT my class"
- The classifier closest to zero (least negative) is:
  - Least unsure about rejecting the point
  - Closest to its boundary on the negative side
  - The "best bad option“, it is the class that rejects this point least strongly
- We still can apply the same rule

$$\hat{y} = \underset{c=0,1,\ldots C-1}{\operatorname{argmax}} \; x^T w_c$$

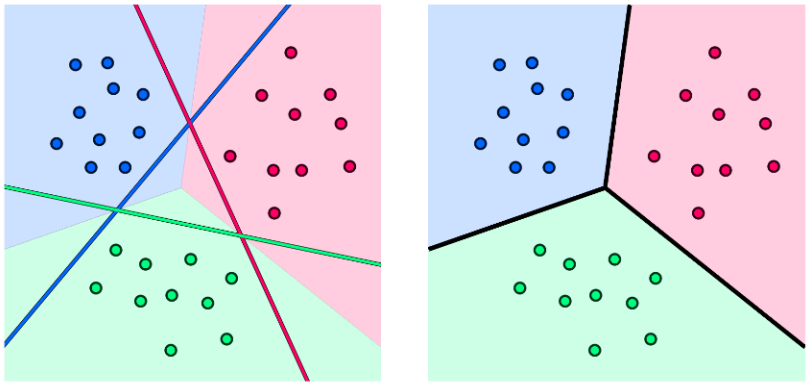
Putting It All Together

- We now have deduced the single rule for assigning a prediction label $\hat{y}$ to a point x:

$$\hat{y} = \underset{c=0,1,\ldots C-1}{\operatorname{argmax}} \; x^T w_c$$

- We call this the **fusion rule** since it tells us precisely how to fuse our C classifiers together to make a unified and consistent classification across the entire space of any dataset.

Multiclass decision boundary



OvA Multi-Class Classifier

- In the previous slides, we train 3 two-class classifiers first, and then use a fusion rule to fuse the results

FUSION RULE

$$\hat{y} = \underset{c=0,1,\ldots C-1}{\operatorname{argmax}} \; x^T w_c$$

- Can we train all 3 classifiers simultaneously to directly satisfy the fusion rule?

Multi-class classification and the Perceptron

- Recall the fusion rule:

$$\hat{y}_p = \underset{c=0,1,\ldots,C-1}{\operatorname{argmax}} \; x_p^T w_c$$

- Our aim is now as follows: instead of tuning each of our classifier's weights one-by-one and then combining them, let's try to learn all sets of weights **simultaneously** so as to satisfy this ideal condition as much as possible. To be specific, let us rewrite the fusion rule slightly as follows:

Distance to true class boundary

$$x_p^T w_{y_p} = \max_{c=0,1,\ldots,C-1} x_p^T w_c$$

Distance to all boundaries

Where w_{y_p} is the **weight vector corresponding to that true class** (we already have this from GT)

- Subtracting $x_p^T w_{y_p}$ from both sides gives us a point-wise cost function that is **always non-negative** and **minimal at zero.**

$$g_p(w_0, \ldots, w_{c-1}) = [(\boxed{\max_{c=0,1,\ldots,C-1} x_p^T w_c}) - \boxed{x_p^T w_{y_p}}]$$

This is the best model prediction it currently has using the current weights over each classifier c

This is model prediction to the true class classifier using the current weights of the true classifier

Loss: Multi-Class Perceptron Function

- Taking the average of this point-wise cost over the entire dataset:

$$g(w_0, \dots, w_{c-1}) = -\frac{1}{P}\sum_{p=1}^{P}[(\max_{c=0,1,\dots,C-1} x_p^T w_c) - x_p^T w_{y_p}]$$

which gives us the **multi-class perceptron cost function**. It has these properties:

- it is convex
- it has a trivial solution at zero, which is often avoided by initializing local optimization away from the origin.

$$loss/cost = g(w_0, \dots, w_{c-1}) = -\frac{1}{P}\sum_{p=1}^{P}\left(\max_{c=0,\dots,C-1} f_{w_c}(x_p) - f_{w_{y_p}}(x_p)\right)$$

Note: in some textbooks, **loss** refers to the difference between actual and predicted value for a single example, whereas in some other textbooks **cost** refers to the aggregate difference over a group of examples (batch). However, for our purposes, these can be used interchangeably.

Example

Using the loss function:

$$loss = g(w_0, \dots, w_{c-1}) = -\frac{1}{P}\sum_{p=1}^{P}\left(\max_{c=0,\dots,C-1} f_{w_c}(x_p) - f_{w_{y_p}}(x_p)\right)$$

Cracker	3.2	1.3	2.2
Mug	5.1	4.9	2.5
Sugar	-1.7	2.0	-3.1
Point-wise loss	1.9	0	5.6

Each of these numbers is the $f_{w_c}(x_p)$

Therefore, taking the average of all 3 examples, the overall loss:

= -1/3 [(max(3.2, 5.1, - 1.7) – 3.2)
+ (max(1.3, 4.9, 2) – 4.9)
+ (max(2.2, 2.5, -3.1) – (-3.1))]

= -1/3 [(5.1 – 3,2) + (4.9 – 4.9) + (2.5 – (-3.1))]

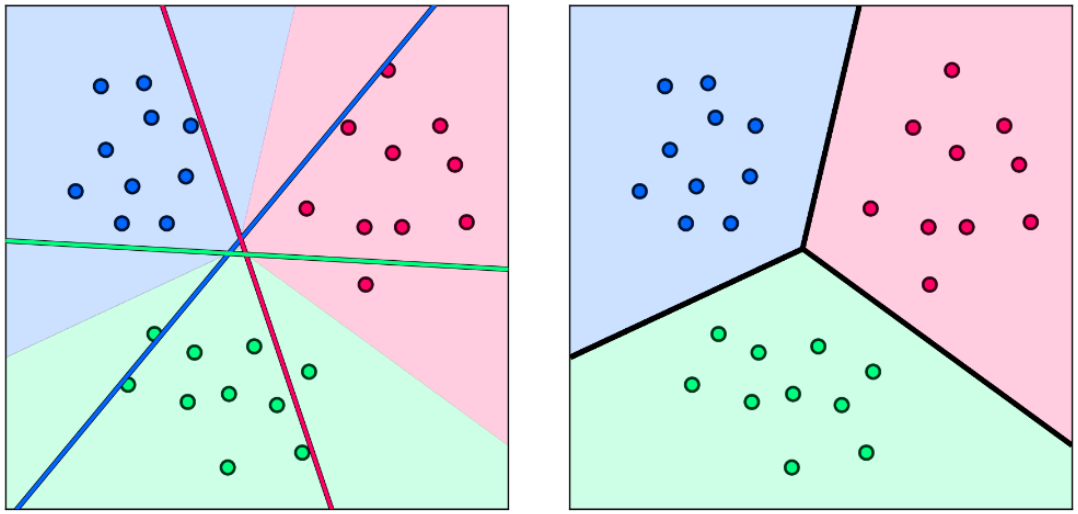
= -1/3 [1.9 + 0 + 5.6]

= -1/3 [7.5] = **-2.5**

Example

- Multi-class on a toy dataset with C=3 classes

Gradient descent (200 iterations, $\alpha = 10^{-2}$)



- On the left, we did not train each individual two-class classifier in a 1va, hence each classifier performs poorly on its data. This is fine because the fused 3 will perform very well as seen on the right

Loss: Softmax Regression

- The multi-class perceptron cost uses the max function, which is not differentiable at the point where two values are equal. This makes optimization difficult.
- Also, the multi-class perceptron cost function is not smooth
- We need to replace it with a smoother function.

- We can use the LogSumExp approximation, which is a smooth approximation of the max function:

$$\max\ \{ s_0, s_1, \dots, s_{C-1} \} \approx \mathrm{LogSumExp}(s_0, s_1, \dots, s_{C-1}) \approx \log(\ e^{s_0} + e^{s_1} + \cdots + e^{s_{C-1}})$$

- We are still dealing with the case of more than two classes, i.e. $y^{(i)} \in \{0,\dots,C-1\}$ where C is the number of classes. Therefore, Softmax regression (multi-class classification; C > 2) is a **generalization** of logistic regression (binary classification; C = 2).

- Now our training set of P labeled examples looks like $\{(\boldsymbol{x_1}, \boldsymbol{y_1}), (\boldsymbol{x_2}, \boldsymbol{y_2}), \dots, (\boldsymbol{x_P}, \boldsymbol{y_P})\}$
  - Each input $\boldsymbol{x_i} \in R^N$ i.e. each $\boldsymbol{x_i}$ is an N-dimensional vector representing N features;
  - Each output $\boldsymbol{y_i} \in \{0,\dots,C-1\}$ i.e. each $\boldsymbol{y_i}$ is a discrete scalar value between 0 to C-1. For example, in MNIST digit recognition, we have C = 10 different classes.

The Multi-Class Softmax Loss

- From Before: we know that:

$$Loss = -\frac{1}{P}\sum_{p=1}^{P}\left(\max_{c=0,\ldots,C-1} f_{w_c}(x_p) - f_{w_{y_p}}(x_p)\right)$$

- Replace max function with $\mathrm{LogSumExp}$
- This leads to the cost function

$$\text{Loss} = -\frac{1}{P}\sum_{p=1}^{P}\left(\log\left(\sum_{c=0}^{C-1} e^{f_{w_c}(x_p)}\right) - f_{w_{y_p}}(x_p)\right)$$

$$= -\frac{1}{P}\sum log\ \left(e^{f_{w_c}(x_p)} / \sum_{P=1}^{P} e^{f_{w_{y_p}}(x_p)}\right)$$

$$= -\frac{1}{P}\sum log\ (\text{softmax}(f_{w_c}(x_p)) \quad \text{// see next slide}$$

- This is called the multi-class softmax loss (aka *multi-class cross entropy cost, multi-class logistic cost)*
- This loss function is convex and smooth, and hence can be easily minimized using local minimization methods

Why It is Called Softmax

- We want to estimate the **probability** of the class label taking on each of the C different possible values. Thus, the final output of the classifier should be a **C-dimensional** vector, whose **elements sum to 1.** This will give us the estimated probabilities for each class.

- To output a probability distribution, we can use the **Softmax** function, which takes as input some C-dimensional vector, say z:

$$softmax(z_i) = \frac{e^{z_i}}{\sum_{j=0}^{C-1} e^{z_j}}$$

  - where each z_i represents **one element** in the input vector z.
  - In our case, each $z_i = x^T w_i$.

- In the denominator, we sum over **all elements** of the input vector z. Dividing by this sum normalizes values in the output vector to be between 0 and 1, which makes it a probability distribution!

The Workflow

- Compute scores: $f_{w_c}(x_p)$ for all classes C
- Softmax is implicitly applied through the ratio

$$e^{f_{w_c}(x_p)} / \sum_{c=0}^{C-1} e^{f_{w_c}(x_p)}$$

  - (in this case, the z from previous slide is $z = f_{w_c}(x_p)$)
- Take negative log of that probability
- Average over all P datapoints

Example of Output of Softmax Regression Activation Function

- Using the earlier Cracker-Mug-Sugar classification example, let us consider the first training example, with this vector: z = [3.2, 5.1, -1.7]^T and C = 3.

- As a reminder, this vector is just an example, but in a real 3-class classifier it would be obtained by taking dot products with 3 sets of weights and the inputs. For example: $z_1 = x^T w_1 = 3.2$, $z_2 = x^T w_2 = 5.1$, and so on.

- Applying $softmax(z)$, we get:

inputs x (represented as some column vector with N features)

$$\xrightarrow{z_i = x^T w_i} \begin{bmatrix} 3.2 \\ 5.1 \\ -1.7 \end{bmatrix} \xrightarrow{\exp(z_i)} \begin{bmatrix} 24.5 \\ 164.0 \\ 5.5 \end{bmatrix} \xrightarrow[24.5 + 164.0 + 5.5]{\text{divide each term by}} \begin{bmatrix} 0.13 \\ 0.84 \\ 0.03 \end{bmatrix}$$

unnormalized outputs z (K-dimensional vector, sometimes referred to as **logits**)

all values are now positive

normalized outputs f(W, x), (probabilities between 0-1 for each class)

Note: in some textbooks, the dot product is taken as ${w_i}^T x$ instead of $x^T w_i$. This is just a difference in notation, and in both cases, the result is a scalar.

Loss: Binary Cross Entropy

$$loss = -\frac{1}{P}\sum_{p} log\,(\text{softmax}(f_{w_c}(x_p)) = -\frac{1}{P}\sum \log(\frac{e^{f_{w_c}(x_p)}}{\sum_{c=0}^{C-1} e^{f_{w_c}(x_p)}})$$

For C=2 classes

$$loss = -\frac{1}{P}\sum y_p \log(f(\mathrm{w},x_p)) - (1-y_p)\log(1-f(\mathrm{w},x_p))$$

In the binary case, the output is a scalar value which is the predicted probability that the p-th training example is a positive example (i.e. its label is 1). Simply put:

- $f(\mathrm{w},x_p)= P(y_p=1|x_p,\mathrm{w})$ //probability that this class is 1
- $1-f(\mathrm{w},x_p)= P(y_p=0|x_p,\mathrm{w})$ //probability that this class is 0

Therefore, the Binary Cross Entropy loss function from earlier can be rewritten as:

$$loss -\frac{1}{P}\sum y_p\ P(y_p=1|x_p,\mathrm{w}) +(1-y_p)\ P(y_p=0|x_p,\mathrm{w})$$

The reason we multiply each term by y_p and $1-y_p$ is because y_p can either be 0 or 1, which means only one of the two additive terms will be non-zero

Generalized Cross Entropy Loss

- We can use similar logic to handle the multi-class case where y_p is in {0,1 , 2,…C-1} as follows:

$$g(w) = -\frac{1}{P}\sum_{p=1}^{P}\sum_{c=0}^{C-1} 1\{y_p = c\}\log P(y_p = c|x_p, w_c)$$

- Note: $1\{\cdot\}$ is called an "indicator function" which evaluates to 1 when the condition in the curly brackets is true, and 0 otherwise.

- Recall that the output of multi-class classification is a C-dimensional vector, where the c-th element is the predicted probability of the input x_p belonging to the c-th class. Simply put:

$$P\left(y_p = c \,\middle|\, x_p, w\right) = \text{the c-th element of } f(w, x_p)$$

Example

- Suppose we're classifying handwritten digits (0-9), so C=10. After feeding an input x_p through the model:

```
f(W, xₚ) = [0.01, 0.05, 0.02, 0.70, 0.03, 0.08, 0.04, 0.02, 0.03, 0.02]
              ↑     ↑     ↑     ↑     ↑     ↑     ↑     ↑     ↑     ↑
            digit digit digit digit digit digit digit digit digit digit
              0     1     2     3     4     5     6     7     8     9
```

- Interpretation:
  - The model is 70% confident this is digit "3“ compared to low probabilities on other digits
- Making the Final Prediction:
  - We pick the class with the highest probability
  - Predicted class = 3 (because 0.70 is the largest probability)
- Connection to the Loss Function:
  - We are computing (− log(probability assigned to the true class))
    - If true class is 3 and model predicts 0.70 → loss = −log(0.70) ≈ 0.36 (small loss, good)
    - If true class is 3 and model predicts 0.01 → loss = −log(0.01) ≈ 4.6- (large loss, bad)

One-Hot Encoding

- Connection to one-Hot Encoding:
  - For a true class $y_p = 3$ in a $C = 5$ problem

```
One-hot vector = [0, 0, 1, 0, 0]
                  ↑  ↑  ↑  ↑  ↑
                class class class class class
                  1    2    3    4    5
```

  - Why useful?
    - In multi-class classification, we are comparing TWO C-dimensional vectors:

```
yₚ (true) = [0, 0, 1, 0, 0]  ← class 3 is the true label
```

```
f(W, xₚ) = [0.05, 0.10, 0.70, 0.10, 0.05]  ← model's prediction
            P(y=1) P(y=2) P(y=3) P(y=4) P(y=5)
```

    - Only the term where c=y contributes to the loss. Others are zero'd out
  - **The Cross-Entropy Loss compares these two vectors:**

$$\boldsymbol{loss = -\frac{1}{P}\sum trueLabel(c) \times log(prediction(c))}$$

  - **In this example**

```
Loss = - [0×log(0.05) + 0×log(0.10) + 1×log(0.70) + 0×log(0.10) + 0×log(0.05)]
     = - [0 + 0 + log(0.70) + 0 + 0]
     = -log(0.70)
```

This is exactly what the indicator function does

$$g(w) = -1/P \sum_p \sum_c 1\{y_p = c\} \log P(y_p = c \mid x_p, w_c)$$

Is equivalent to:

$$g(w) = -1/P \sum_p \sum_c \text{one_hot}[c] \times \log(\text{predicted}[c])$$

Where $1\{y_p = c\}$ is the one-hot encoding!

Logistic Regression vs Softmax Regression

Feature	Logistic Regression	Softmax Regression
Number of Classes	Binary ($C = 2$)	Multi-class ($C \geq 2$)
Output Type	Scalar (single probability)	C-dimensional vector
Activation Function $f(w,x)$	Sigmoid: $\sigma(x^T w)$ where $\sigma(z) = 1 / (1 + e^{-z})$ where $z = x^T w$	Softmax: $\text{softmax}(z) = e^z / \Sigma e^{zj}$ where $z = x^T w$
Prediction	$P(y = 1 \mid x, w) = \sigma(x^T w)$ $P(y = 0 \mid x, w) = 1 - \sigma(x^T w)$	$P(y = c \mid x, w) = e^{x^T w} / \Sigma e^{x^T w}$
Parameters	One weight vector $w \in \mathbb{R}^{N+1}$	C weight vectors (matrix $W \in \mathbb{R}^{C \times N+1}$)
Loss Function	Binary Cross-Entropy: $\text{Loss} = -(1/P) \Sigma [y_p \log(\sigma(x^T w)) + (1-y_p) \log(1-\sigma(x^T w))]$	Multi-class Cross-Entropy: $\text{Loss} = -(1/P) \Sigma \Sigma \mathbb{1}\{y_p=c\} \log P(y_p=c \mid x_p, w_k)$
Decision Rule	Predict class 1 if $P(y=1 \mid x,w) > 0.5$	Predict class k where $k = \text{argmax}_k P(y=c \mid x,w)$
Decision Boundary	One hyperplane	C hyperplanes
Key Relationship	Softmax is a generalization of logistic regression. When $c = 2$, softmax reduces to the sigmoid function.	

Softmax Regression Formulae

- Combining the linear step (dot product of weights and inputs) with the softmax function, we get the following classifier function:

$$f(W,x) = \frac{1}{\sum_{j=0}^{C-1} e^{w_j^T x}} \begin{bmatrix} e^{w_0^T x} \\ e^{w_1^T x} \\ \vdots \\ e^{w_{C-1}^T x} \end{bmatrix}$$

- Generalized Cross Entropy loss function:

$$g(w) = -\frac{1}{P} \sum_{p=1}^{P} \sum_{c=0}^{C-1} 1\{y_p = c\} \log\left(\frac{e^{w_c^T x_p}}{\sum_{i=0}^{C-1} e^{w_i^T x_p}}\right)$$

where c is the c'th class label, and w_c is the weight vector corresponding to the weights from the c'th class classifier.

Property of Softmax Regression

- Softmax regression has the property of redundant set of parameters. This means that if we take our parameters w_i $(i = 0,1, \ldots C-1)$ and we subtract a fixed vector ψ from each of them such that each w_i becomes w_i - ψ, then this does not change our predictions $f(w, x)$
- In other words, if the cost function $g(w)$ is minimized over the weights $w = \{w_0, \ldots w_{C-1}\}$, it is equivalent to minimizing over the weights $w' = \{w_0 - \psi, \ldots w_{C-1} - \psi\}$.
- This is because the softmax function is invariant to constants being added or subtracted to the weights.
- Proof: $P'(y = c \mid x) = \frac{e^{(w_c+\psi)^T x}}{\sum_j e^{\left(w_j+\psi\right)^T x}} = \frac{e^{w_c^T x} . e^{\psi^T x}}{\sum_j e^{w_j^T x} . e^{\psi^T x}} = \frac{e^{w_c^T x}}{\sum_j e^{w_j^T x}} = P(y = c \mid x)$

Softmax Regression is a Generalization of Logistic Regression

- $f(w,x) = \text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=0}^{C-1} e^{z_j}}$ where $z_i = x^T w_i$

- Plugging in C = 2 in the Softmax formula: $f(w,x) = \frac{1}{e^{x^T w_1} + e^{x^T w_2}} \begin{bmatrix} e^{x^T w_1} \\ e^{x^T w_2} \end{bmatrix}$

- Now we subtract w_1 from both sets of weights, and using the redundant parameter property of softmax regression, we get:

$$f(W,x) = \frac{1}{e^0 + e^{x^T(w_2 - w_1)}} \begin{bmatrix} e^0 \\ e^{x^T(w_2 - w_1)} \end{bmatrix}$$

$$= \begin{bmatrix} \frac{1}{1 + e^{x^T(w_2 - w_1)}} \\ \frac{x^T e^{(w_2 - w_1)}}{1 + e^{x^T(w_2 - w_1)}} \end{bmatrix}$$

- Finally, setting $w' = w_2 - w_1$ and recalling the formula for the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$, we get $f(w,x) = \begin{bmatrix} \sigma(x^T w') \\ 1 - \sigma(x^T w^T) \end{bmatrix}$

  Probability of class 1

  Probability of class 0

- This is the same output as logistic regression, except in logistic regression, instead of the full vector, we output only the first scalar $\sigma(x^T w^T)$, and the other term is implicit!

- From previous slide, Softmax regression output (with C=2)

$(z) = \frac{1}{1+e^{-z}}$, we get $f(w,x) = \begin{bmatrix} \sigma(x^T w') \\ 1 - \sigma(x^T\ w^T) \end{bmatrix}$ Probability of class 1 / Probability of class 0

- **Recall that the logistic regression output:**

$$f(w,x) = \sigma(x^T\ w')$$ //single scalar showing the probability of one class

  - $if\ \sigma(x^T\ w') \geq 0.5$ predict class 1
  - $if\ \sigma(x^T\ w') < 0.5$ predict class 0
  - $\mathrm{P(class\ 1)} = \sigma(x^T\ w')\ //\ explicit$
  - $\mathrm{P(class\ 1)} = \sigma(x^T\ w')\ //\ implicit$

- Hence the output of Softmax is the same as logistic regression, except in logistic regression, instead of the full vector, we output only the first scalar $\sigma(x^T\ w^T)$, and the other term is implicit.